

TUGAS 3

II4021 – KRIPTOGRAFI



Disusun oleh:

Brandon Theodore Ferrinov	18223020
Rafli Dwi Nugraha	18223038
Ahmad Evander Ruizhi Xavier	18223064

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI.....	1
DAFTAR GAMBAR.....	3
DAFTAR TABEL.....	4
PERNYATAAN.....	5
BAB I	
TEORI SINGKAT.....	6
1.1 JSON Web Token (JWT).....	6
1.2 Elliptic Curve Diffie-Hellman (ECDH).....	6
1.3 Hash-based Message Authentication Code (HMAC).....	7
1.4 HMAC-based Key Derivation Function (HKDF).....	7
1.5 Advanced Encryption Standard (AES).....	8
1.6 Unit Testing.....	8
1.7 Docker.....	8
BAB II	
PERANCANGAN DAN IMPLEMENTASI.....	10
2.1 Lingkungan Pengembangan dan Kakas.....	10
2.1.1 Arsitektur Perangkat Lunak.....	10
2.1.2 Kakas Sisi Klien.....	10
2.1.3 Kakas Sisi Server.....	11
2.1.4 Penyimpanan dan Basis Data.....	11
2.2 Perancangan dan Implementasi Manajemen Kunci (ECDH, HKDF, dan Storage Key).....	11
2.3 Perancangan dan Implementasi Enkripsi Pesan (AES).....	13
2.4 Perancangan dan Implementasi Autentikasi Sesi (JWT).....	14
2.5 Perancangan Antarmuka Pengguna.....	19
2.6 Bonus.....	24
2.6.1 Perancangan dan Implementasi Autentikasi dan Integritas Pesan (HMAC). 24	
2.6.2 Unit Testing.....	25
2.6.3 Docker.....	25
BAB III	
PENGUJIAN PROGRAM DAN ANALISIS HASIL.....	26

3.1 Uji Autentikasi dan Manajemen Pengguna.....	26
3.2 Uji Implementasi JSON Web Token (JWT).....	27
3.3 Uji Pembentukan Kunci Komunikasi (ECDH dan HKDF).....	29
3.4 Uji Enkripsi dan Dekripsi Pesan (AES).....	32
3.5 Uji Integritas dan Autentikasi Pesan (HMAC).....	35
BAB IV	
KESIMPULAN.....	39
DAFTAR PUSTAKA.....	40
LAMPIRAN.....	41

DAFTAR GAMBAR

Gambar 2.1 Tampilan antarmuka halaman Register.....	20
Gambar 2.2 Tampilan antarmuka halaman Login.....	21
Gambar 2.3 Tampilan antarmuka halaman Chat sebelum memilih kontak.....	22
Gambar 2.3 Tampilan antarmuka halaman Chat sebelum memilih kontak.....	23

DAFTAR TABEL

Tabel 3.1 Skenario pengujian autentikasi dan manajemen pengguna.....	26
Tabel 3.2 Skenario pengujian implementasi JWT.....	27
Tabel 3.3 Skenario pengujian ECDH dan HKDF.....	29
Tabel 3.4 Skenario pengujian enkripsi dan dekripsi pesan.....	32
Tabel 3.5 Skenario pengujian integritas dan autentikasi pesan.....	35

PERNYATAAN

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (*raw output*) dari *Generative AI*, melainkan hasil pengembangan dan penulisan mandiri.



Brandon Theodore
Ferrinov



Rafli Dwi Nugraha



Ahmad Evander Ruizhi
Xavier

BAB I

TEORI SINGKAT

1.1 JSON Web Token (JWT)

JSON Web Token (JWT) merupakan standar protokol (RFC 7519) yang memungkinkan pengiriman informasi dalam format objek JSON dengan ringkas. JWT berperan sebagai alat autentikasi yang menjamin keamanan informasi melalui tanda tangan digital, baik menggunakan *shared secret* maupun pasangan kunci publik atau privat. JWT berfokus dalam penggunaan aspek *signed tokens* untuk menjamin integritas data agar tidak dimanipulasi dan memastikan keaslian informasi hanya berasal dari pihak yang memiliki kunci sah. Mekanisme ini dapat memverifikasi identitas pengguna tanpa membutuhkan penyimpanan data sesi di sisi server.

1.2 Elliptic Curve Diffie-Hellman (ECDH)

Elliptic Curve Diffie-Hellman (ECDH) adalah protokol pertukaran kunci yang memungkinkan dua pihak untuk menyepakati sebuah kunci rahasia bersama (*shared secret*) melalui saluran komunikasi publik. Berbeda dengan algoritma Diffie-Hellman biasa, ECDH menggunakan sifat matematis perkalian titik pada kurva eliptik yang memungkinkan pengirim dan penerima hanya menukarkan kunci publik mereka untuk kemudian dikombinasikan dengan kunci privat masing-masing. Hasil akhir dari proses ini akan menghasilkan nilai rahasia yang identik di kedua belah pihak tanpa harus mengirimkan kunci rahasia tersebut secara langsung sehingga algoritma ini sangat efektif dalam membangun jalur enkripsi yang aman dan efisien secara komputasi.

1.3 Hash-based Message Authentication Code (HMAC)

HMAC adalah algoritma untuk menghasilkan kode autentikasi pesan yang mengombinasikan fungsi *hash* kriptografi (misalnya SHA-256) dengan kunci rahasia. HMAC memberikan jaminan keamanan yang lebih kuat terhadap manipulasi data karena hasil *hash* dikombinasikan dengan kunci rahasia (*secret key*) sehingga memiliki sifat kriptografis yang tidak dapat dibalikkan dan tahan terhadap *collision*. Dalam implementasinya untuk keamanan, HMAC digunakan untuk memastikan pesan berasal dari pengirim yang sah (*authenticity*), menjamin pesan yang dikirim tidak diubah oleh pihak tak berwenang (*integrity*) dan bisa juga digunakan untuk penurunan kunci (*key derivation*). Implementasi HMAC umumnya menggunakan pendekatan kriptografi Encrypt-then-MAC yaitu dengan mengenkripsi plainteks terlebih dahulu dan kemudian menggunakan cipherteks tersebut untuk menghitung nilai MAC. Pada saat pesan diterima, cipherteks diverifikasi terlebih dahulu sebelum proses dekripsi dilakukan."

1.4 HMAC-based Key Derivation Function (HKDF)

HKDF adalah fungsi penurunan kunci yang memanfaatkan mekanisme HMAC untuk mengubah kunci mentah yang pendek atau belum terlalu teracak (seperti *shared secret* hasil ECDH) menjadi kunci kriptografi yang kuat dan berukuran tetap. HKDF bekerja melalui dua tahapan utama yaitu *extract* dengan menghasilkan kunci acak yang padat dan *expand* dengan memperluas kunci tersebut menjadi beberapa kunci turunan (seperti kunci AES dan HMAC). HKDF sangat efektif dan efisien dalam protokol komunikasi untuk menghasilkan kunci sesi yang aman dari *shared secret* hasil dari pertukaran kunci ECDH.

1.5 Advanced Encryption Standard (AES)

Advanced Encryption Standard (AES) adalah algoritma kriptografi kunci simetris berbasis *block cipher* yang menjadi standar global dalam mengenkripsi data. AES bekerja dengan mengenkripsi data dalam blok berukuran 128 bit menggunakan panjang kunci 128, 192, atau 256-bit dengan menggunakan jumlah putaran operasi (10, 12, atau 14 putaran) yang ditentukan oleh panjang kunci yang dipilih. Setiap putaran dalam AES secara umum menggunakan beberapa operasi kompleks seperti Sub Bytes, Shift Rows, Mix Columns, dan Add Round Key. Meskipun lebih sulit untuk diimplementasikan, AES memiliki efisiensi dan kekuatan keamanan yang jauh lebih baik dibandingkan standar lama seperti DES dan triple DES. AES menjadi standar utama yang digunakan saat ini dalam mengamankan komunikasi internet, melindungi data sensitif, dan enkripsi file di seluruh dunia.

1.6 Unit Testing

Unit testing adalah salah satu jenis pengujian software yang digunakan untuk menguji unit - unit terkecil dalam sebuah perangkat lunak. Unit testing biasanya digunakan untuk menguji fungsi, method, ataupun class. Tujuan unit testing diadakan adalah untuk mendeteksi bug secara dini, menulis kode yang lebih baik, dan menghasilkan dokumentasi. Dalam tugas ini, unit testing diimplementasikan menggunakan vitest karena sesuai dengan runtime yang digunakan yaitu bun.

1.7 Docker

Docker merupakan virtualisasi atau kontainerisasi dari sebuah platform yang bekerja pada level sistem operasi yang memungkinkan aplikasi untuk berjalan pada sistem operasi host dan bukan pada sistem operasi lain. Docker menyelesaikan sebuah masalah besar pada proses deployment tradisional yaitu suatu sistem yang

terkadang hanya bekerja pada mesin tertentu. Docker menyelesaikan masalah tersebut dengan menstandarkan lingkungan runtime dengan cara mem-*bundling* kode aplikasi dengan dependensi nya masing masing menjadi satu unit.

BAB II

PERANCANGAN DAN IMPLEMENTASI

2.1 Lingkungan Pengembangan dan Kakas

2.1.1 Arsitektur Perangkat Lunak

Sistem ini menggunakan arsitektur Client-Server berbasis *web* yaitu dengan antarmuka pengguna berjalan pada *web* menggunakan HTML, CSS, dan JavaScript untuk menangani proses kriptografi secara lokal melalui Web Crypto API. Sisi klien berinteraksi dengan Server melalui protokol HTTP untuk manajemen basis data dan pengiriman pesan terenkripsi dan server berperan sebagai perantara yang tidak memiliki akses terhadap kunci privat maupun pesan asli. Model ini memastikan beban pemrosesan data terenkripsi dilakukan di sisi pengguna dan server hanya bertanggung jawab terhadap ketersediaan data dan sinkronisasi komunikasi antar pengguna.

2.1.2 Kakas Sisi Klien

Kakas sisi klien dibuat dengan menggunakan teknologi standar pengembangan *web* yaitu HTML5 untuk pembangunan antarmuka, CSS3 untuk desain antarmuka, dan JavaScript sebagai logika utama kriptografi pada sistem. Proses kriptografi pada klien dikelola secara modular melalui file `/client/js/crypto/ecdh.js` dan `/client/js/crypto/aes_hmac.js` yang memanfaatkan Web Crypto API. Semua komponen ini dikelola melalui `/client/js/main.js` yang mengatur alur interaksi pengguna dan komunikasi asinkron dengan server menggunakan metode Fetch API.

2.1.3 Kakas Sisi Server

Pada server digunakan framework [Elysia](#) JS sebagai backend untuk integrasi antara database dengan front-end. Elysia JS merupakan framework backend untuk Typescript. Framework ini dipilih karena cukup mudah untuk diimplementasikan untuk routing API dan integrasi.

2.1.4 Penyimpanan dan Basis Data

users	messages
id	id
email	sender_email
password	recipient_email
created_at	iv
publickey	ciphertext
encrypted_pk	mac
salt	created_at
iv	
mac	

Database yang digunakan untuk aplikasi ini adalah PostgreSQL. Database sederhana ini terdiri dari 2 tabel users dan messages. tabel users akan menyimpan data akun yang sudah terdaftar dengan password yang sudah di-*hash* dan juga *private key* yang sudah terenkripsi. Lalu tabel messages akan menyimpan seluruh *chat* antara 2 akun.

2.2 Perancangan dan Implementasi Manajemen Kunci (ECDH, HKDF, dan Storage Key)

Perancangan dan implementasi manajemen kunci pada sistem ini menggunakan kombinasi dari ECDH berbasis X25519, HKDF, serta storage key berbasis PBKDF2 untuk

memastikan keamanan komunikasi dan penyimpanan data. Pada proses pertukaran kunci, sistem menggunakan algoritma Elliptic Curve Diffie-Hellman (ECDH) dengan kurva X25519 melalui fungsi `generateKeyPair()` untuk menghasilkan pasangan public key dan private key. Public key kemudian dapat diekspor menggunakan format raw, sedangkan private key diekspor dalam format PKCS8 agar dapat disimpan atau didistribusikan secara aman. Setelah dua pihak saling bertukar public key, fungsi `deriveSharedKey()` digunakan untuk menghasilkan shared secret melalui proses `deriveBits` dengan panjang 256 bit. Dengan pendekatan ini, kedua pihak dapat memperoleh secret key yang sama tanpa perlu mengirimkan kunci rahasia secara langsung melalui jaringan.

Shared secret yang dihasilkan dari proses ECDH tidak langsung digunakan sebagai kunci enkripsi, tetapi diproses kembali menggunakan HKDF (HMAC-based Key Derivation Function) melalui fungsi `deriveAESKey()`. Pada tahap ini, shared secret terlebih dahulu diimpor sebagai HKDF key, kemudian dilakukan derivasi menggunakan hash SHA-256, salt acak sepanjang 16 byte, dan parameter info "aes-key". Hasil derivasi tersebut digunakan untuk membentuk kunci AES-CBC 256-bit yang nantinya dipakai dalam proses enkripsi dan dekripsi data.

Selain untuk komunikasi, sistem juga menerapkan mekanisme storage key untuk melindungi data yang disimpan secara lokal. Fungsi `generateStorageKey()` memanfaatkan PBKDF2 dengan hash SHA-256 dan iterasi sebanyak 10000 untuk melakukan derivasi kunci dari password pengguna dengan salt yang diberikan. Hasil derivasi menghasilkan 512 bit data yang kemudian dibagi menjadi dua bagian, yaitu 256 bit pertama untuk membentuk AES key dan 256 bit sisanya untuk membentuk HMAC key. AES key digunakan untuk menjaga kerahasiaan data melalui enkripsi

AES-CBC, sedangkan HMAC key digunakan untuk memastikan integritas dan autentikasi data melalui mekanisme tanda tangan digital berbasis HMAC-SHA256.

2.3 Perancangan dan Implementasi Enkripsi Pesan (AES)

Implementasi kerahasiaan data dalam sistem ini menggunakan algoritma AES dengan panjang kunci 256 bit dalam mode operasi Cipher Block Chaining (CBC). Seluruh logika enkripsi (dan dekripsi) diimplementasikan pada file `/client/js/crypto/aes_hmac.js`. Panjang kunci 256 bit pada AES-256 merupakan standar keamanan tingkat tinggi yang memberikan perlindungan yang sangat kuat terhadap serangan *brute force*. Pemilihan mode CBC sebagai varian AES yang digunakan bertujuan untuk memastikan setiap blok data saling bergantung sehingga tidak ada pola yang terlihat pada hasil enkripsi meskipun terdapat bagian pesan yang berulang. Varian AES lain seperti CTR tidak dipilih karena setiap blok tidak saling bergantung dan memiliki urutan sehingga berisiko tinggi jika terjadi duplikasi pada nilai Initialization Vector (IV). Sementara itu, varian GCM tidak dipilih untuk menghindari redundansi fitur keamanan (MAC) karena GCM sudah memiliki autentikasi bawaan yang berisiko tumpang tindih dengan implementasi HMAC manual sebagai poin bonus pada tugas ini.

Dalam proses implementasinya, fungsi `encryptAndSign` pada file `/client/js/crypto/aes_hmac.js` merupakan fungsi utama yang mengelola seluruh alur pengamanan pesan sebelum dikirim. Fungsi ini bertanggung jawab untuk mengubah plainteks menjadi nilai biner, membangkitkan IV unik sebesar 16 byte menggunakan `crypto.getRandomValues`, dan mengeksekusi algoritma enkripsi AES-CBC melalui metode `crypto.subtle.encrypt`. Fungsi ini juga mengintegrasikan mekanisme autentikasi dengan menghitung nilai MAC sebelum dikemas menjadi

paket data untuk dikirim. Di sisi penerima, fungsi `verifyAndDecrypt` melakukan proses pemulihan data dengan menerima parameter IV dan cipherteks dalam format Base64. Cipherteks tersebut kemudian dikonversi kembali menjadi `ArrayBuffer` melalui fungsi pembantu `_base64ToArrayBuffer` dan teks asli dikembalikan menggunakan metode `crypto.subtle.decrypt`. Fungsi ini akan menangkap kesalahan dan melempar pesan error jika terdapat ketidaksesuaian pada kunci atau IV. Proses verifikasi tersebut memastikan hanya data valid dengan parameter enkripsi yang tepat yang dapat diproses lebih lanjut.

2.4 Perancangan dan Implementasi Autentikasi Sesi (JWT)

Implementasi Json Web Token dilakukan pada `/server/app/src/lib/jwt.js`. Dalam proses implementasi, dibuat 4 error yang dapat di-*throw* jika terjadi kesalahan.

```
class JWTError extends Error {}
class InvalidTokenError extends JWTError {}
class InvalidSignError extends JWTError {}
class ExpiredSignError extends JWTError {}
```

`JWTError` merupakan exception yang di-*throw* ketika terdapat parameter yang tidak diperbolehkan, header tidak memiliki parameter 'alg', header tidak memiliki parameter 'typ', format token bukan JWT, dan penggunaan algoritma yang tidak didukung. `InvalidTokenError` di-*throw* apabila format JWT salah (Tidak terdiri dari 3 bagian) ketika JWT tidak bisa di decode. `InvalidSignError` akan di-*throw* jika signature tidak valid. Sementara `ExpiredSignError` akan di-*throw* apabila signature sudah expired sesuai ketentuan yang diberikan pada claims.

Algoritma yang dapat digunakan juga disimpan dalam dictionary bernama `algorithm` yang menyimpan algoritma signing yang dapat digunakan. Berikut merupakan algoritma yang dapat digunakan pada tugas ini.

```
const algorithm = {
  ES256: {
    hash: "sha256",
    curve: "prime256v1",
    size: 32,
  },

  ES384: {
    hash: "sha384",
    curve: "secp384r1",
    size: 48,
  },

  ES512: {
    hash: "sha512",
    curve: "secp521r1",
    size: 66,
  },
};
```

Dapat dilihat bahwa terdapat 3 algoritma yang dapat digunakan yaitu ES256, ES384, dan ES512.

Dalam file jwt.js terdapat 2 fungsi utama yang digunakan sebagai *core process* dari implementasi ini yaitu fungsi sign dan verify. Fungsi sign berguna untuk menghasilkan JWT, struktur JWT yang digunakan terdiri dari header, payload, dan signature yang dibungkus dalam format base64. Berikut merupakan contoh payload yang digunakan dalam tugas ini

```
const token = sign(
  {
    alg: "ES256",
    typ: "JWT"
  },
```



```

{
  iss: "Evan Raja Kripto",
  sub: "Rafli",
  aud: "chat",
  iat: Math.floor(Date.now() / 1000),
  nbf: Math.floor(Date.now() / 1000) ,
  exp: Math.floor(Date.now() / 1000) + 3600,
},

{
  message: "Halo siapa nama kamu"
},
pemPrivate
);

```

Pada implementasi ini, fungsi sign menerima 4 parameter yaitu header, claims, payload, dan privKey. Struktur fungsi sign juga cukup sederhana, pertama dilakukan sebuah pengecekan apakah header sudah sesuai, lalu dilanjutkan dengan pengecekan algoritma enkripsi yang digunakan. Lalu header dan payload akan di encode dalam format base64url sebelum akhirnya di sign dan dikembalikan oleh fungsi sign. Berikut implementasi utama dari fungsi ini

```

const config = algorithm[alg];

const fullPayload = {
  ...payload,
  ...claims,
};

const encodedHeader = urlEncode(jsonEncode(header));

const encodedPayload = urlEncode(jsonEncode(fullPayload));

const headerPayload = `${encodedHeader}.${encodedPayload}`;

const signature = crypto.sign(config.hash, Buffer.from(headerPayload),
{

```

```

    key: privKey,
    dsaEncoding: "ieee-p1363",
  });

  const encodedSignature = urlEncode(signature);

  return `${encodedHeader}.` + `${encodedPayload}.` +
    `${encodedSignature}`;

```

Selain fungsi sign, terdapat fungsi verify yang dapat digunakan untuk memverifikasi JWT, fungsi ini berguna untuk memastikan apakah JWT sudah sesuai. Fungsi verify menerima 3 buah parameter yaitu jwt, publicKey, dan options. Options merupakan parameter yang digunakan untuk memverifikasi algoritma yang digunakan dan beberapa parameter pada claim khususnya iss, sub, dan aud.

Proses verifikasi pada fungsi ini dilakukan dalam beberapa tahap, pertama dilakukan verifikasi terhadap format JWT, dengan kode berikut

```

const parts = jwt.split(".");

if (parts.length !== 3) {
  throw new InvalidTokenError("Format JWT Salah");
}

```

Lalu JWT akan dipecah menjadi 3 bagian dan dilakukan pengecekan terhadap header

```

const [encodedHeader, encodedPayload, encodedSignature] = parts;

let header;
let payload;

try {
  header = jsonDecode(urlDecode(encodedHeader));

  payload = jsonDecode(urlDecode(encodedPayload));
}

```

```

    } catch {
        throw new InvalidTokenError("JWT Invalid");
    }

    const alg = header.alg;

    if (!(alg in algorithm)) {
        throw new JWTErrror("Algoritma tidak disupport");
    }

    if (options.algs && !options.algs.includes(alg)) {
        throw new JWTErrror("Algoritma tidak diperbolehkan");
    }

```

Setelah itu akan dilakukan pengecekan terhadap signature

```

const valid = crypto.verify(
    config.hash,
    Buffer.from(headerPayload),
    {
        key: publicKey,
        dsaEncoding: "ieee-p1363",
    },
    signature,
);

if (!valid) {
    throw new InvalidSignError("Signature Invalid");
}

```

Terakhir merupakan pengecekan terhadap claim

```

if (options.iss && payload.iss !== options.iss) {
    throw new JWTErrror("iss Invalid");
}

if (options.sub && payload.sub !== options.sub) {
    throw new JWTErrror("sub Invalid");
}

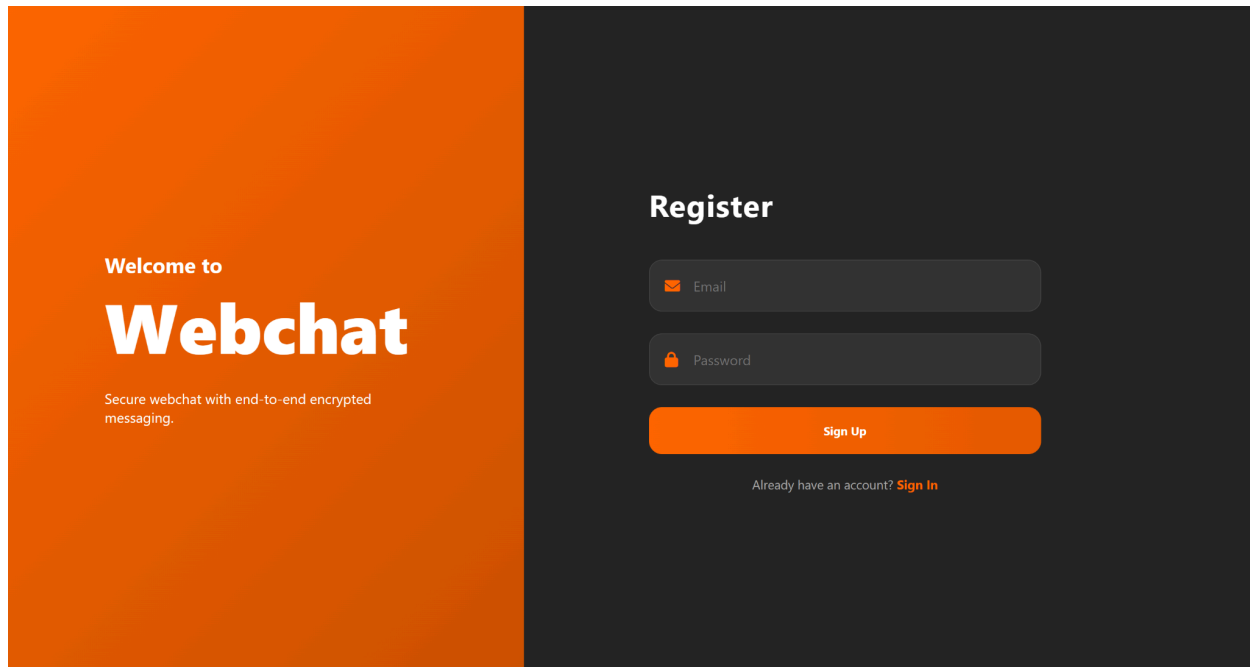
```

```
if (options.aud && payload.aud !== options.aud) {  
    throw new JWTError("aud Invalid");  
}  
  
if (options.jti && payload.jti !== options.jti) {  
    throw new JWTError("jti invalid");  
}
```

Terakhir jika JWT sudah berhasil melewati semua tahap verifikasi, fungsi verify akan mengembalikan header, payload, dan signature yang sudah di decode..

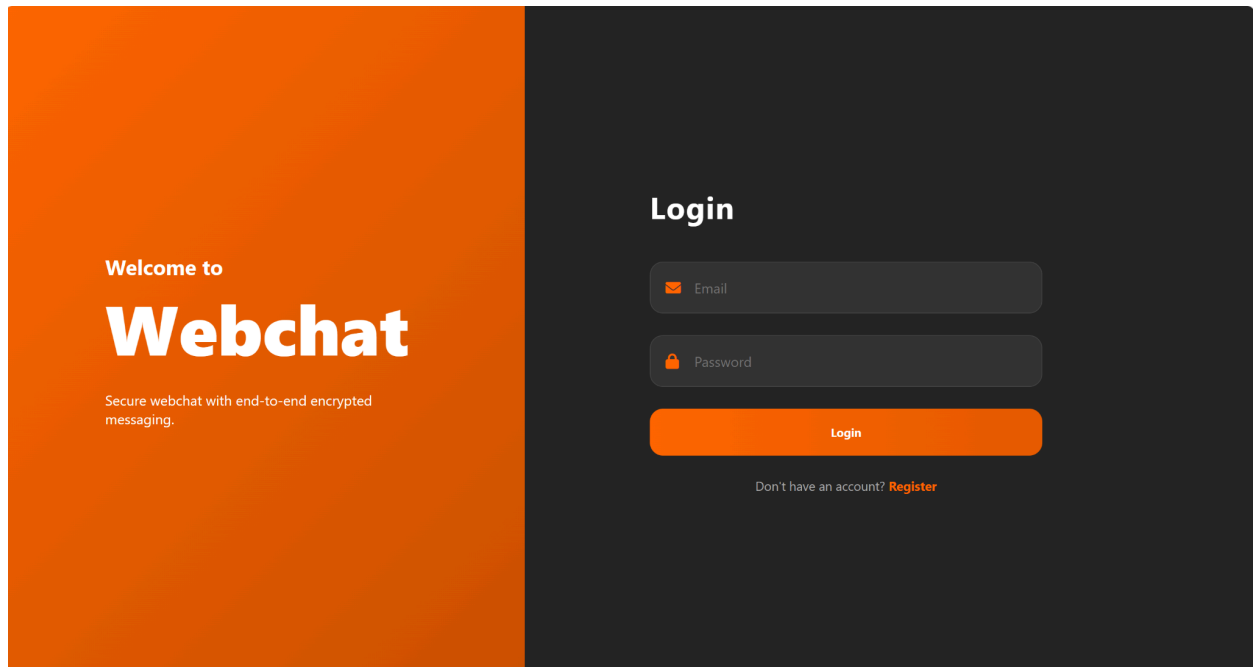
2.5 Perancangan Antarmuka Pengguna

Perancangan antarmuka pengguna pada menggunakan konsep Single Page Application (SPA) yang berpusat sepenuhnya pada satu berkas index.html utama. Seluruh transisi dari halaman Register, Login, dan Chat dikelola secara dinamis melalui manipulasi DOM menggunakan JavaScript untuk memberikan pengalaman pengguna yang responsif. Pendekatan ini meminimalkan *reload* dan memastikan seluruh algoritma kriptografi di sisi klien tetap terjaga dalam satu halaman eksekusi yang konsisten. Berikut ini adalah tampilan seluruh halaman beserta penjelasan singkat untuk setiap komponen dan tombol navigasi.



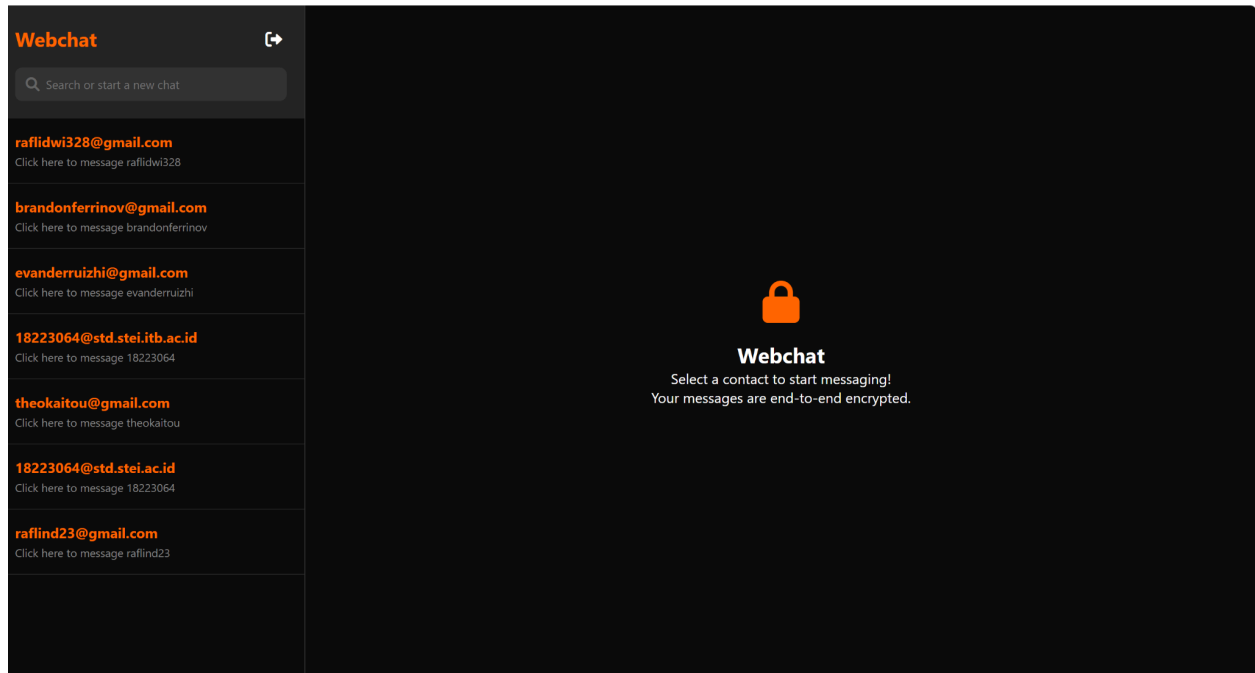
Gambar 2.1 Tampilan antarmuka halaman Register

Halaman Register memiliki dua komponen utama yaitu kolom pengisian email dan *password* dan satu tombol navigasi utama yaitu tombol Sign Up. Teks input yang dimasukkan pengguna ke kolom *password* akan disembunyikan untuk menjamin keamanannya. Tombol navigasi Sign Up berfungsi untuk melakukan registrasi pengguna dan hanya bisa dilakukan jika seluruh kolom terisi dan akun email registrasi tidak sama. Jika pengguna sudah memiliki akun, terdapat tombol Sign In untuk mengakses halaman Login tanpa melakukan registrasi



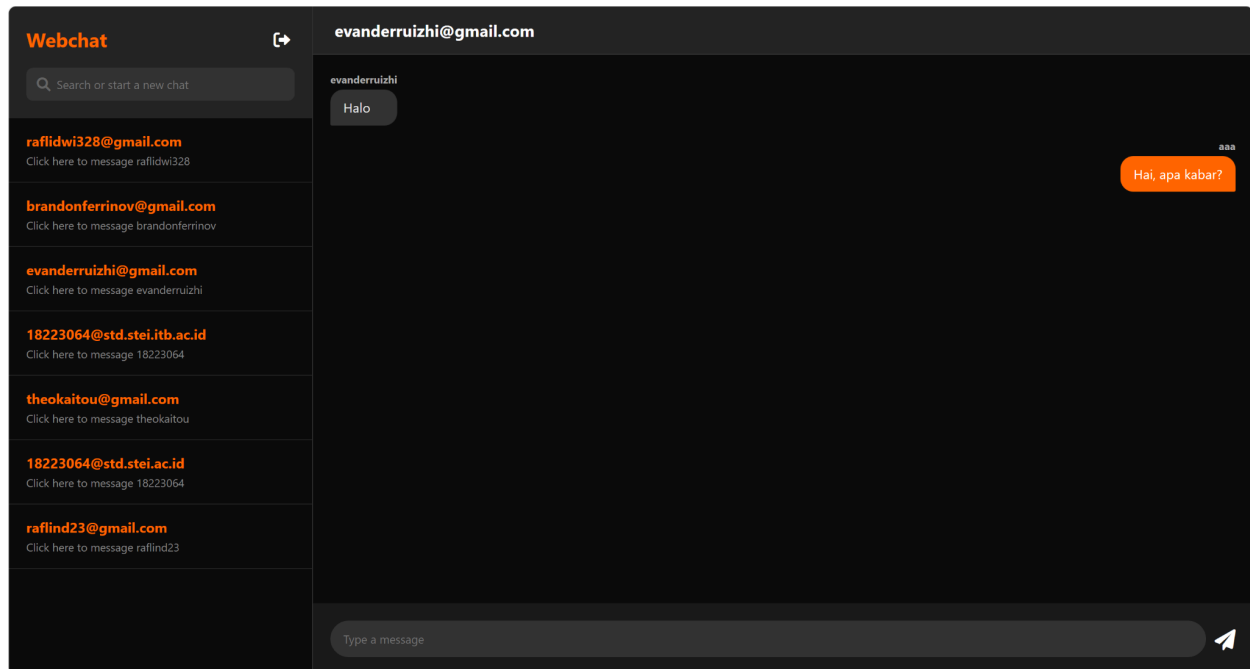
Gambar 2.2 Tampilan antarmuka halaman Login

Halaman Login memiliki desain yang sama seperti halaman Register dan hanya berbeda di bagian tombol navigasi yang digunakan. Tombol Login digunakan untuk mengakses Webchat dengan menggunakan email beserta *password* yang valid dan akses akan ditolak jika terdapat ketidaksesuaian input pada salah satu atau kedua kolom. Jika belum memiliki akun, disediakan tombol Register untuk navigasi ke halaman Register dan melakukan registrasi.



Gambar 2.3 Tampilan antarmuka halaman Chat sebelum memilih kontak

Jika pengguna sudah melakukan login, maka navigasi dilanjutkan ke Halaman Chat yang menampilkan seluruh kontak yang sudah terdaftar pada Webchat (kecuali kontak sendiri) dengan layar *chat* masih belum menampilkan ruangan *chat* (hanya berupa pesan statis). Seluruh daftar kontak tersedia di sebelah kanan halaman dan pengguna dapat mengakses kontak manapun untuk melakukan percakapan dengan pengguna email terkait.



Gambar 2.3 Tampilan antarmuka halaman Chat sebelum memilih kontak

Jika kontak sudah dipilih, maka sisi kanan halaman Chat akan menampilkan ruangan *chat* dengan pengguna email terkait. Pengguna dapat mengetik pesan melalui kolom pesan di bagian bawah layar dan mengirimkan pesan tersebut melalui ikon kirim di sebelah kanan kolom pesan. Pesan yang dikirim dan diterima dapat dilihat melalui *bubble chat* di ruangan *chat* dengan *bubble chat* warna jingga sebagai penanda diri pengguna sendiri (orang pertama) yang sedang menggunakan Webchat dan *bubble chat* warna hitam sebagai penanda pengguna lain (orang kedua) yang menjadi tujuan pengiriman pesan orang pertama. *Username* kedua pihak juga ditampilkan di atas *bubble chat* dan *username* itu sendiri adalah email yang dipotong mulai dari karakter '@' (misal evanderruizhi@gmail.com akan memiliki *username* evanderruizhi). Hal ini memungkinkan penggunaan *username* tanpa konfigurasi tambahan dan penyimpanan spesifik pada basis data. Pengguna dapat melakukan logout dengan menekan ikon keluar yang terdapat di bagian kiri atas halaman.

2.6 Bonus

2.6.1 Perancangan dan Implementasi Autentikasi dan Integritas Pesan (HMAC)

Sistem mengimplementasikan HMAC dengan fungsi *hash* SHA-256 pada *file* `/client/js/crypto/aes_hmac.js`. Mekanisme ini dirancang menggunakan skema Encrypt-then-MAC yang menggunakan kalkulasi kode autentikasi pada data yang telah dienkripsi. Pendekatan ini memungkinkan sistem untuk memvalidasi integritas paket data sebelum melakukan proses dekripsi. Pemilihan HMAC-SHA256 didasarkan pada ketahanannya terhadap serangan manipulasi data dan kemampuannya dalam memastikan pesan memang berasal dari pengirim yang memiliki kunci autentikasi yang sah.

Dalam implementasinya, fungsi `encryptAndSign` pada *file* `/client/js/crypto/aes_hmac.js` menghitung nilai MAC dengan menggabungkan parameter IV dan ciphertext sebagai satu *buffer* melalui fungsi pembantu `_appendBuffer`. Hasil penggabungan ini kemudian ditandatangani menggunakan metode `crypto.subtle.sign`. Di sisi penerima, fungsi `verifyAndDecrypt` menjalankan proses verifikasi menggunakan metode `crypto.subtle.verify` untuk membandingkan nilai MAC yang diterima dengan hasil perhitungan ulang. Jika hasilnya tidak identik, fungsi ini akan langsung melempar pesan error dan menghentikan proses sebelum tahap dekripsi dimulai. Struktur *payload* yang mengintegrasikan IV, ciphertext, dan MAC dalam format Base64 memastikan seluruh komponen pengamanan ditransmisikan secara utuh dalam satu kesatuan paket data.

2.6.2 Unit Testing

Unit testing dilakukan menggunakan framework **vitest**, file testing dapat dilihat pada **/server/app/src/lib/jwt.test.js**. Framework vitest digunakan sebab implementasi kelompok kami sudah menggunakan bun, sehingga vitest merupakan pilihan yang paling tepat untuk digunakan sebagai testing framework.

Test yang dilakukan dalam unit test ini meliputi happy path test, 4 edge case pada fungsi sign berupa; parameter algoritma tidak ada, penggunaan algoritma selain ES256, ES384, dan ES512, parameter typ yang salah, dan parameter header yang kelebihan, lalu 3 edge case pada fungsi verify yang meliputi; signature salah, token expired, dan parameter aud pada claim salah.

2.6.3 Docker

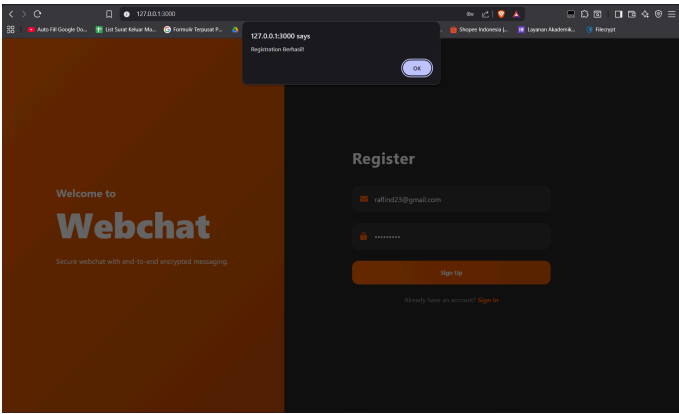
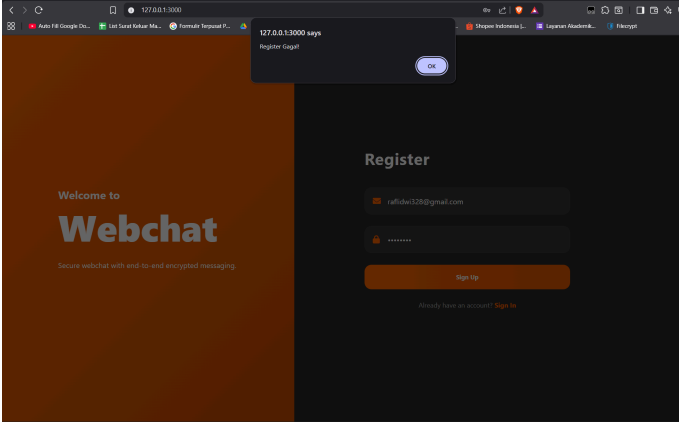
Dalam tugas ini, diimplementasikan docker untuk memudahkan penjalanan sistem di setiap device. Terdapat 1 docker image yang di-*build* yang berisikan keseluruhan sistem. Dockerfile diletakkan pada folder **/app** karena program harus dijalankan pada folder yang memuat package.json, namun penjalanan docker tetap dilakukan pada directory utama. Untuk menjalankan layanan docker, sudah disediakan script bernama run.sh yang ketika dijalankan akan langsung mem-*build* dan me-*run* docker.

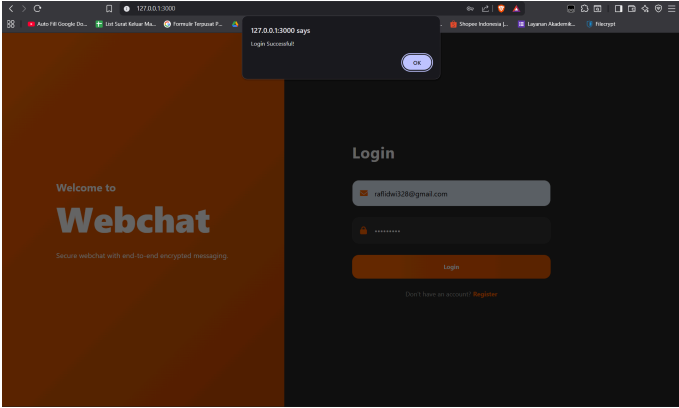
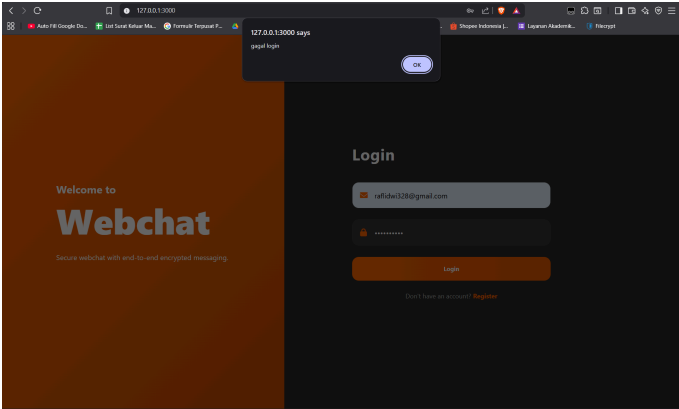
BAB III

PENGUJIAN PROGRAM DAN ANALISIS HASIL

3.1 Uji Autentikasi dan Manajemen Pengguna

Tabel 3.1 Skenario pengujian autentikasi dan manajemen pengguna

No	Skenario Pengujian	Langkah-Langkah	Hasil Pengujian	Status
1.	Registrasi dengan data valid	1. Lakukan register dengan email yang belum terdaftar		Berhasil
2.	Registrasi dengan email yang sudah terdaftar	1. Lakukan register dengan email yang sudah terdaftar		Berhasil

3.	Login dengan <i>password</i> yang benar	1. Lakukan login dengan akun yang sudah terdaftar dan password valid		Berhasil
4.	Login dengan <i>password</i> yang salah	1. Lakukan login dengan data yang tidak valid		Berhasil

Dari hasil pengujian untuk implementasi Autitentikasi user, dapat dilihat bahwa semua hasil pengujian menunjukkan seluruh skenario pengujian telah berhasil.

3.2 Uji Implementasi JSON Web Token (JWT)

Seluruh pengujian dilakukan menggunakan sebuah kode yang khusus disiapkan sesuai dengan kasus uji. Kode bisa dilihat pada bagian bawah file jwt.js yang sudah diubah dalam bentuk komentar.

Tabel 3.2 Skenario pengujian implementasi JWT

No	Skenario Pengujian	Langkah-Langkah	Hasil Pengujian	Status
----	--------------------	-----------------	-----------------	--------

1.	Menggunakan fungsi <i>sign</i> untuk menghasilkan JWT, lalu melakukan <i>verify</i> dengan <i>public key</i> yang benar	1. Jalankan <i>jwt.js</i> tanpa ada perubahan	<pre> JWT: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJtZXNzYWdlIjoiaS9GfSbyBzaWwYSWY1hGthbXU1LCJpc3MiOiJFdmFuIFJhamEgS3JpcHRvIiwic3ViIjoiaW50bGkiLCJhdW0iOiJjaGFiIiwiaWF0IjoxNzc4NTc4OTAwLCJyYmY1OjE3ZnZg1NzkgSMDAsImV4cCI6MTc3ODU0MjUwMH0.Fqh78x0rD2GNDc-wIqGT2IAG2Rh60lWPm6M9NnE4Rf1Yh2WR58LMQch4WT-SxSwRyALrV189pHiU4sqEQNS1w DECODED: { header: { alg: 'ES256', typ: 'JWT' }, payload: { message: 'Halo siapa nama kamu', iss: 'Evan Raja Kripto', sub: 'Rafli', aud: 'chat', iat: 1778578988, nbf: 1778578988, exp: 1778582588 }, signature: 'Fqh78x0rD2GNDc-wIqGT2IAG2Rh60lWPm6M9NnE4Rf1Yh2WR58LMQch4WT-SxSwRyALrV189pHiU4sqEQNS1w' } </pre>	Berhasil
2.	Melakukan <i>verify</i> dengan <i>public key</i> yang salah, perhatikan bahwa verifikasi gagal	1. Ubah variabel <i>pemPublic</i> pada <i>decode</i> menjadi <i>pemPublic2</i> 2. Jalankan file <i>jwt.js</i>	<pre> [theokaitou@fedora]~/Documents/Tubes3Kripto/server/app/src/lib% node jwt.js JWT: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJtZXNzYWdlIjoiaS9GfSbyBzaWwYSWY1hGthbXU1LCJpc3MiOiJFdmFuIFJhamEgS3JpcHRvIiwic3ViIjoiaW50bGkiLCJhdW0iOiJjaGFiIiwiaWF0IjoxNzc4NTc4OTAwLCJyYmY1OjE3ZnZg1NzkgSMDAsImV4cCI6MTc3ODU0MjUwMH0.TZw5Uh8AXPNRvAwPj8xkUCXDVfTHdgplonjiPmDiDqbM0gSlo58az_p0estEqH0tKtZkT2nnxolh5R0NH--Dw file:///home/theokaitou/Documents/Tubes3Kripto/server/app/src/lib/jwt.js:154 throw new InvalidSignatureError("Signature Invalid"); ^ InvalidSignatureError: Signature Invalid at verify (file:///home/theokaitou/Documents/Tubes3Kripto/server/app/src/lib/jwt.js:154:11) at file:///home/theokaitou/Documents/Tubes3Kripto/server/app/src/lib/jwt.js:260:17 at ModuleJob.run (node:internal/modules/esm/module_job:343:25) at async onImport.tracePromise.__proto__ (node:internal/modules/esm/loader:665:26) at async asyncRunEntryPointWithESMLoader (node:internal/modules/run_main:117:5) Node.js v22.2.0 </pre>	Berhasil
3.	Menguji token dengan format tidak valid, perhatikan bahwa sistem melempar error	1. Hilangkan parameter <i>typ</i> pada header token 2. Jalankan file <i>jwt.js</i>	<pre> [theokaitou@fedora]~/Documents/Tubes3Kripto/server/app/src/lib% node jwt.js file:///home/theokaitou/Documents/Tubes3Kripto/server/app/src/lib/jwt.js:71 throw new JWTError("typ tidak ditemukan"); ^ JWTError: typ tidak ditemukan at sign (file:///home/theokaitou/Documents/Tubes3Kripto/server/app/src/lib/jwt.js:71:11) at file:///home/theokaitou/Documents/Tubes3Kripto/server/app/src/lib/jwt.js:232:15 at ModuleJob.run (node:internal/modules/esm/module_job:343:25) at async onImport.tracePromise.__proto__ (node:internal/modules/esm/loader:665:26) at async asyncRunEntryPointWithESMLoader (node:internal/modules/run_main:117:5) Node.js v22.2.0 </pre>	Berhasil
4.	Menguji token dengan klaim waktu (<i>exp/nbf</i>) yang tidak sesuai	1. Hilangkan <i>+</i> 3600 pada parameter <i>exp</i> token 2. Jalankan file <i>jwt.js</i>	<pre> [theokaitou@fedora]~/Documents/Tubes3Kripto/server/app/src/lib% node jwt.js JWT: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJtZXNzYWdlIjoiaS9GfSbyBzaWwYSWY1hGthbXU1LCJpc3MiOiJFdmFuIFJhamEgS3JpcHRvIiwic3ViIjoiaW50bGkiLCJhdW0iOiJjaGFiIiwiaWF0IjoxNzc4NTc4OTAwLCJyYmY1OjE3ZnZg1NzkgSMDAsImV4cCI6MTc3ODU0MjUwMH0.TTZxgHK9MfQa_Fz1Ser0Y6b0r7jj18P5yl_rQx18dYUNZR28WeMopzSGr8vsE2ITZfwsL7_VEE6VTrnZHQ0B0w file:///home/theokaitou/Documents/Tubes3Kripto/server/app/src/lib/jwt.js:162 throw new ExpiredSignatureError("Token expired"); ^ ExpiredSignatureError: Token expired at verify (file:///home/theokaitou/Documents/Tubes3Kripto/server/app/src/lib/jwt.js:162:15) at file:///home/theokaitou/Documents/Tubes3Kripto/server/app/src/lib/jwt.js:259:17 at ModuleJob.run (node:internal/modules/esm/module_job:343:25) at async onImport.tracePromise.__proto__ (node:internal/modules/esm/loader:665:26) at async asyncRunEntryPointWithESMLoader (node:internal/modules/run_main:117:5) Node.js v22.2.0 </pre>	Berhasil

Bonus	Unit Testing	1. Jalankan file <code>jwt.test.js</code> dengan command <code>"bun test"</code>	<pre> Happy Path Tests > JWT berhasil di sign Happy Path Tests > Verify JWT berhasil Edge Case sign > Gaada alg Edge Case sign > Algoritma gak support Edge Case sign > typ nya salah Edge Case sign > Parameter kelebihan Edge Cases verify > Signature salah [1.00ms] Edge Cases verify > Token expired Edge Cases verify > aud salah 9 pass 0 fail 11 expect() calls Ran 9 tests across 1 file. [17.00ms] </pre>	Berhasil
-------	--------------	--	---	----------

Berdasarkan hasil pengujian JWT diatas, dapat dilihat bahwa 4 uji kasus berhasil dijalankan dengan baik. Selain itu spesifikasi bonus untuk unit testing juga berhasil dijalankan dengan 2 happy path test, 4 edge case untuk fungsi sign, dan 3 edge case untuk fungsi verify.

3.3 Uji Pembentukan Kunci Komunikasi (ECDH dan HKDF)

Tabel 3.3 Skenario pengujian ECDH dan HKDF

No	Skenario Pengujian	Langkah-Langkah	Hasil Pengujian	Status
1.	Melakukan <i>key exchange</i> untuk dua pengguna	1. Buka aplikasi di dua browser berbeda (misal: Chrome dan Edge) dan login dengan akun yang berbeda. Disini diasumsikan user A login pada browser A dan user B login pada browser B 2. Pada salah	<pre> evanderruizhi@gmail.com: main.js:278 Private Key: MC4CAQAwBQ... main.js:279 Target Public Key: oAB/UQGzd6... main.js:280 PENGUJIAN ECDH: main.js:281 Shared Secret (Base64): main.js:282 5UKEyjmOL7EGq1Asg/Hyf/B2HYaUPkn9RrWIBJCusBo= 18223064@std.stei.itb.ac.id: main.js:278 Private Key: MC4CAQAwBQ... main.js:279 Target Public Key: wk0ynrY1js... main.js:280 PENGUJIAN ECDH: main.js:281 Shared Secret (Base64): main.js:282 5UKEyjmOL7EGq1Asg/Hyf/B2HYaUPkn9RrWIBJCusBo= </pre>	Berhasil

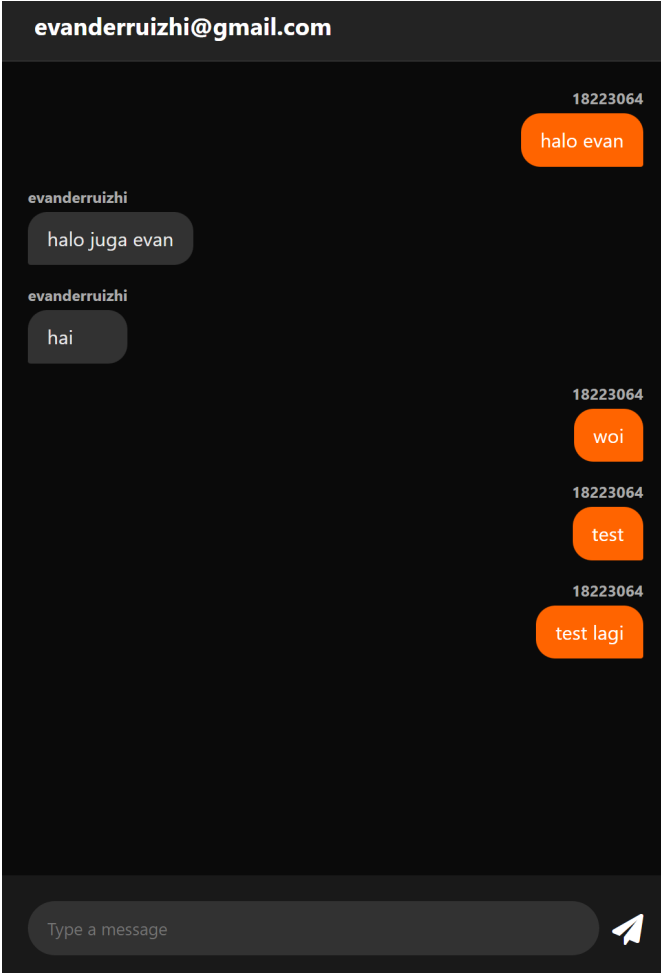
		satu akun, klik nama kontak akun yang login pada browser lain (misal user A memilih kontak user B pada browser A) 3. Lakukan hal yang sama seperti tahap 2 untuk user lainnya (user B memilih kontak user A pada browser B) 4. Buka Console Log masing-masing browser dan bandingkan nilai Shared Secret. Log diharapkan menghasilkan Shared Secret yang sama		
--	--	--	--	--

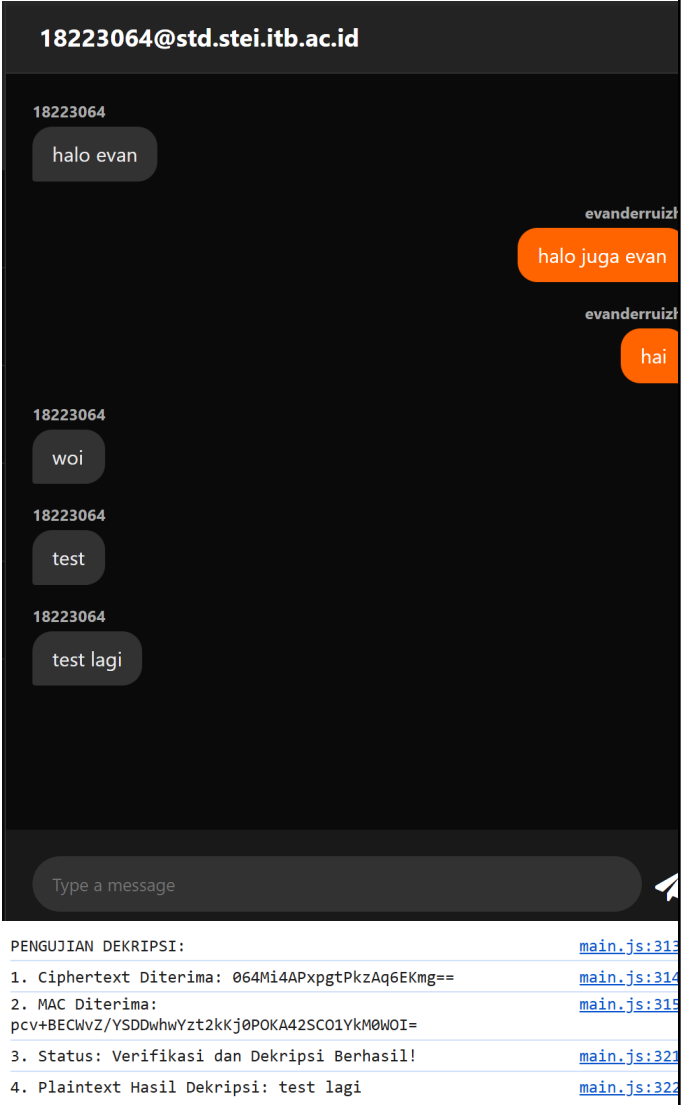
2.	Menunjukkan <i>shared secret</i> berhasil diturunkan menjadi kunci simetris yang digunakan untuk enkripsi pesan	1. Lakukan <i>key exchange</i> seperti skenario pengujian 1 bagian ini 2. Buka Console Log dan lihat keluaran bahwa AES Key dan HMAC Key telah berhasil diturunkan.	<pre> evanderruizhi@gmail.com: main.js:278 Private Key: MC4CAQAwBQ... main.js:279 Target Public Key: oAB/UQGzd6... main.js:280 PENGUJIAN ECDH: main.js:281 Shared Secret (Base64): main.js:282 5UKEyjmOL7EGq1AsG/Hyf/B2HYaUPkn9RrWIBJCusBo= PENGUJIAN KDF: main.js:283 AES Key Generated: main.js:284 ▼ CryptoKey {type: 'secret', extractable: false, algorithm: {...}, usages: Array(2)} ⓘ ▶ algorithm: {name: 'AES-CBC', length: 256} extractable: false type: "secret" ▶ usages: (2) ['encrypt', 'decrypt'] ▶ [[Prototype]]: CryptoKey HMAC Key Generated: main.js:285 ▼ CryptoKey {type: 'secret', extractable: false, algorithm: {...}, usages: Array(2)} ⓘ ▶ algorithm: {name: 'HMAC', hash: {...}, length: 256} extractable: false type: "secret" ▶ usages: (2) ['sign', 'verify'] ▶ [[Prototype]]: CryptoKey 18223064@std.stei.itb.ac.id: main.js:278 Private Key: MC4CAQAwBQ... main.js:279 Target Public Key: wk0ynrY1js... main.js:280 PENGUJIAN ECDH: main.js:281 Shared Secret (Base64): main.js:282 5UKEyjmOL7EGq1AsG/Hyf/B2HYaUPkn9RrWIBJCusBo= PENGUJIAN KDF: main.js:283 AES Key Generated: main.js:284 ▼ CryptoKey {type: 'secret', extractable: false, algorithm: {...}, usages: A rray(2)} ⓘ ▶ algorithm: {name: 'AES-CBC', length: 256} extractable: false type: "secret" ▶ usages: (2) ['encrypt', 'decrypt'] ▶ [[Prototype]]: CryptoKey HMAC Key Generated: main.js:285 ▼ CryptoKey {type: 'secret', extractable: false, algorithm: {...}, usages: A rray(2)} ⓘ ▶ algorithm: {name: 'HMAC', hash: {...}, length: 256} extractable: false type: "secret" ▶ usages: (2) ['sign', 'verify'] ▶ [[Prototype]]: CryptoKey </pre>	Berhasil
----	---	---	---	----------

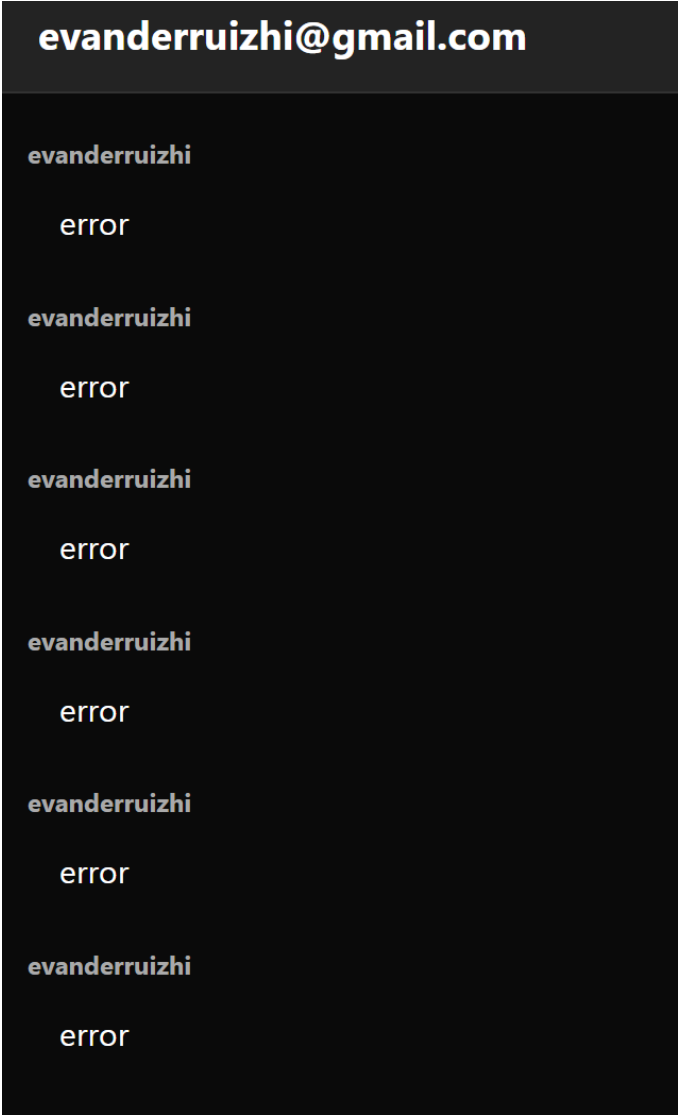
Hasil analisis menunjukkan bahwa algoritma ECDH telah berhasil diimplementasikan dengan benar. Hal tersebut terbukti dari nilai Shared Secret yang identik pada kedua sisi pengguna (5UKEyjm...) meskipun keduanya memiliki kunci privat yang berbeda. Keberhasilan pengujian ini dilanjutkan dengan proses HKDF yang secara otomatis menurunkan Shared Secret tersebut menjadi objek kunci simetris yang spesifik yaitu AES-CBC 256 bit untuk enkripsi dan kunci HMAC untuk proteksi integritas.

3.4 Uji Enkripsi dan Dekripsi Pesan (AES)

Tabel 3.4 Skenario pengujian enkripsi dan dekripsi pesan

No	Skenario Pengujian	Langkah-Langkah	Hasil Pengujian	Status
1.	Mengirim pesan dengan kunci yang benar	- Jalankan program Webchat tanpa mengubah kode program - Login dengan salah satu akun dan pilih target kontak yang ingin dikirimkan pesan - Ketik pesan yang ingin dikirimkan dan tekan tombol kirim untuk mengirim pesan - Login dengan akun yang dikirimkan pesan sebelumnya dan buka kontak pengirim pesan - Lihat pesan yang terkirim melalui <i>bubble</i>	 PENGUJIAN ENKRIPSI: main.js:1 1. Plaintext: test lagi main.js:1 2. IV: 5LGPj5ZP8KER9suZuC/1Nw== main.js:1 3. Ciphertext: 064Mi4APxpgtPkzAq6EKmg== main.js:1 4. MAC: pcv+BECWvZ/YSDDwhwYzt2kKj0POKA42SCO1YkM0WOI= main.js:1	Berhasil

		chat dan pastikan apakah pesan sudah terkirim dengan benar 6. Buka Console Log untuk melihat proses Enkripsi dan Dekripsi dengan lebih detail	 18223064@std.stei.itb.ac.id 18223064 halo evan evanderruizl halo juga evan evanderruizl hai 18223064 woi 18223064 test 18223064 test lagi Type a message PENGUJIAN DEKRIPSI: main.js:313 1. Ciphertext Diterima: 064Mi4APxpgtPkzAq6EKmg== main.js:314 2. MAC Diterima: pcv+BECWvZ/YSDDwhwYzt2kKj0POKA42SC01YkM0W0I= main.js:319 3. Status: Verifikasi dan Dekripsi Berhasil! main.js:322 4. Plaintext Hasil Dekripsi: test lagi main.js:322	
--	--	---	---	--

2.	Menggunakan kunci atau data yang tidak sesuai	1. Buka kode sumber program pada file <code>client/js/main.js</code> dan nyalakan komentar untuk variabel <code>msg.ciphertext</code> (line 312 di dalam fungsi <code>_loadMessages()</code>) untuk mengubah isi cipherteks 2. Login dan buka salah satu kontak yang sudah pernah dikirimkan pesan 3. Lihat tampilan <i>bubble chat</i> dan Console Log untuk menunjukkan bahwa terjadi error dalam proses dekripsi		Berhasil
----	---	--	---	----------

Hasil pengujian menunjukkan bahwa algoritma AES-256 mode CBC berhasil melakukan transformasi plainteks "test lagi" menjadi cipherteks. Konsistensi nilai cipherteks dan IV antara sisi pengirim dan penerima membuktikan bahwa proses enkripsi dan dekripsi berjalan dengan benar jika menggunakan kunci yang sama.

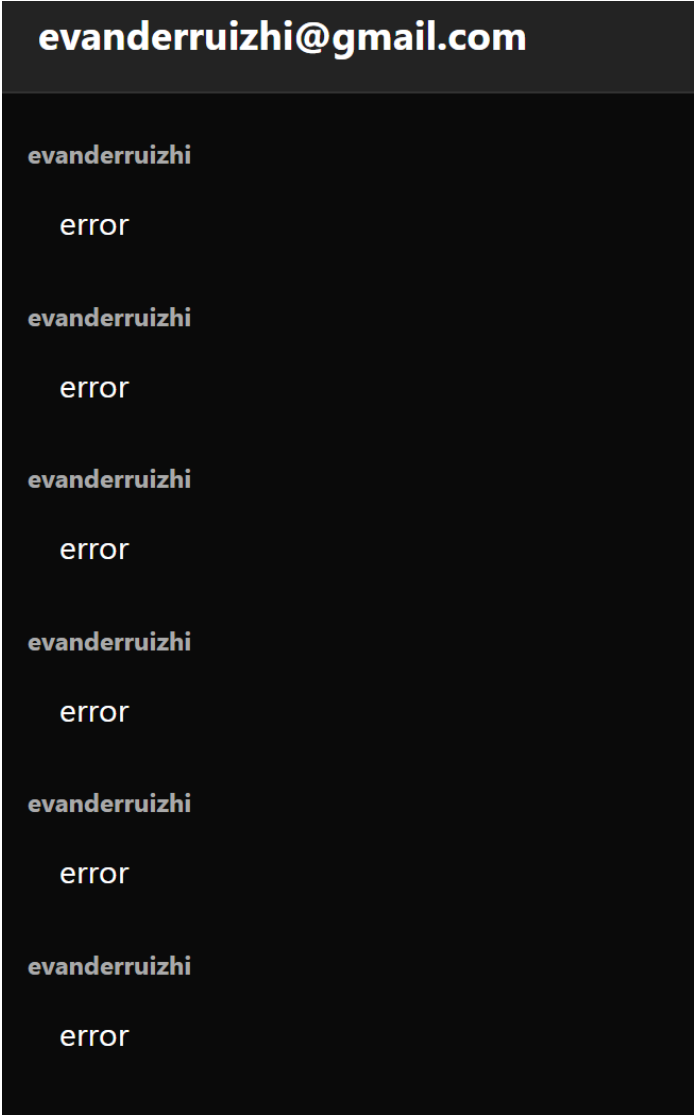
Ketika skenario kegagalan dilakukan dengan memodifikasi cipherteks, sistem menghentikan proses dekripsi dan menampilkan error yang membuktikan bahwa data yang telah berubah tidak dapat dipulihkan kembali ke plainteks aslinya.

3.5 Uji Integritas dan Autentikasi Pesan (HMAC)

Tabel 3.5 Skenario pengujian integritas dan autentikasi pesan

No	Skenario Pengujian	Langkah-Langkah	Hasil Pengujian	Status
1.	Mengirim pesan dengan MAC yang valid	- Jalankan program Webchat tanpa mengubah kode program - Login dengan salah satu akun dan pilih target kontak yang ingin dikirimkan pesan - Ketik pesan yang ingin dikirimkan dan tekan tombol kirim untuk mengirim pesan - Login dengan akun yang dikirimkan pesan sebelumnya dan buka	PENGUJIAN DEKRIPSI: main.js:313 1. Ciphertext Diterima: 064Mi4APxpgtPkzAq6EKmg== main.js:314 2. MAC Diterima: pcv+BECWvZ/YSDDwhwYzt2kKj0POKA42SCO1YkM0W0I= main.js:315 3. Status: Verifikasi dan Dekripsi Berhasil! main.js:321 4. Plaintext Hasil Dekripsi: test lagi main.js:322	Berhasil

		kontak pengirim pesan 5. Lihat pesan yang terkirim melalui <i>bubble chat</i> dan pastikan apakah pesan sudah terkirim dengan benar 6. Buka Console Log untuk melihat proses Enkripsi dan Dekripsi dengan lebih detail		
--	--	---	--	--

2.	Mengirim pesan dengan isi pesan atau MAC yang diubah	1. Buka kode sumber program pada file <code>client/js/main.js</code> dan nyalakan komentar untuk variabel <code>msg.mac</code> (line 311 di dalam fungsi <code>_loadMessages()</code> untuk mengubah nilai MAC 2. Login dan buka salah satu kontak yang sudah pernah dikirimkan pesan 3. Lihat tampilan <i>bubble chat</i> dan Console Log untuk menunjukkan bahwa terjadi error dalam proses verifikasi		Berhasil
----	--	---	---	----------

Pengujian HMAC-SHA256 membuktikan bahwa sistem dapat melakukan validasi autentikasi dan integritas pesan dengan benar sebelum data diproses lebih lanjut. Pada kondisi normal, nilai MAC yang dikirimkan cocok dengan hasil perhitungan

ulang di sisi penerima sehingga status verifikasi dinyatakan berhasil. Namun pada skenario kegagalan di mana data atau MAC dimanipulasi, log pengujian menunjukkan bahwa proses berhenti setelah cipherteks diterima tanpa memunculkan hasil dekripsi. Hal ini mengonfirmasi bahwa mekanisme Verify-then-Decrypt berfungsi dengan baik dengan menolak pesan yang tidak valid atau telah mengalami perubahan pada saat transmisi.

BAB IV

KESIMPULAN

Berdasarkan pengujian yang dilakukan, didapat kesimpulan berikut:

- Seluruh skenario pengujian autentikasi berhasil divalidasi dengan hasil yang sesuai pada setiap tahapan proses masuk pengguna ke dalam sistem.
- Pengujian JWT berhasil melewati seluruh kasus uji utama serta memenuhi spesifikasi unit testing baik pada *happy path* dan *edge case*.
- Implementasi ECDH berhasil menghasilkan *shared secret* yang identik di kedua sisi dan kemudian diturunkan secara akurat melalui HKDF menjadi kunci simetris AES dan HMAC.
- Algoritma AES-256-CBC terbukti efektif menjaga kerahasiaan pesan, proses dekripsi hanya berhasil jika kunci dan cipherteks tetap konsisten tanpa modifikasi.
- Mekanisme Encrypt-then-MAC dan Verify-then-Decrypt menggunakan HMAC-SHA256 berhasil menjamin integritas pesan, proses dekripsi ditolak pada data yang telah dimanipulasi.

DAFTAR PUSTAKA

[Web Crypto API - Web APIs | MDN](#)

[Web Crypto API | Node.js v26.1.0 Documentation](#)

[How to Convert ArrayBuffers to Base64 \(Storing Web Crypto Data\)](#)

[JSON Web Token Introduction - jwt.io](#)

<https://elysiajs.com/table-of-content.html>

[ECDH Key Exchange | Practical Cryptography for Developers](#)

[HMAC and Key Derivation | Practical Cryptography for Developers](#)

[Unit Testing](#)

[What is Docker](#)

LAMPIRAN

1. Pranala repositori

<https://github.com/Raflind/Tubes3Kripto>

2. Pranala video demo

https://drive.google.com/file/d/1e2tbgngZn2C7V7KOOACpC7vIwD_PIsFI/view?usp=sharing

3. Pembagian tugas

NIM	Nama	Kontribusi
18223020	Brandon Theodore Ferrinov	- Implementasi JWT - Bonus Unit Testing - Bonus Docker
18223038	Rafli Dwi Nugraha	- Implementasi ECDH HKDF - Implementasi Fitur Login, Logout, Register - Pembuatan Backend Aplikasi
18223064	Ahmad Evander Ruizhi Xavier	- Implementasi AES-256 mode CBC - Implementasi MAC (HMAC)